COMID

WEEK 04 · 차시 4-1 ~ 4-5 · 50분

LAMMPS 기초와 시스템 구축

LAMMPS 개요와 설치 · 입력 스크립트 해부 · 데이터 파일과 토폴로지 · 초기 구조와 다층 계면 셀 생성

윤태영 (Taeyoung Yun)

실습 · 298 K NPT 평형화

퀴즈 4주차

RECAP · WEEK 03

지난 주차 요약과 이번 주차의 위치

3주차 산출물

힘장(force field) 파라미터가 적용된 분자 라이브러리와 목표 조성의 벌크 셀

남은 질문

신뢰할 수 있는 물성을 얻기 위한 이 셀의 실행 절차

이번 주차

LAMMPS 설치 · 입력 스크립트 · 데이터 파일 · 초기 구조 생성

파이프라인 6단계 중 03 MD 계산의 실행 절차를 확정

OBJECTIVE

4주차 학습목표

01 LAMMPS의 구조와 패키지 체계를 이해하고 CPU · GPU/KOKKOS 빌드를 설치

02 입력 스크립트를 units · atom_style · pair_style · fix · run 단위로 해부하고 직접 작성

03 원자 좌표 · 토폴로지(topology) · 힘장 파라미터로 구성되는 data 파일 구조를 판독

04 Packmol · Moltemplate · RDKit 으로 초기 구조와 부착 · 계면용 다층 셀을 생성

AGENDA

4주차 구성

4-1	LAMMPS 개요와 설치	8분
4-2	입력 스크립트 해부	16분
4-3	데이터 파일과 토폴로지	8분
4-4	초기 구조 생성	10분
4-5	다층구조 생성	8분
부록	재현성 규약 · 작업 제출과 큐 → 13주차 보충 A(13-S2 · 13-S3)에서 다룸	자율 학습

4-1

LAMMPS 개요와 설치

8분 · LAMMPS 개요 · GPU/KOKKOS 빌드 · 패키지 구조

4-1 · OVERVIEW

LAMMPS의 계산 대상

정의

LAMMPS(Large-scale Atomic/Molecular Massively Parallel Simulator) = 입자에 작용하는 힘으로 Newton 방정식을 적분하는 고전 분자동역학(MD, molecular dynamics) 엔진 — 전자 상태 계산 없이 힘장으로 대체

병렬 구조

공간을 쪼개 프로세스에 나눠 주는 영역 분할(domain decomposition) · MPI(Message Passing Interface)로 수십~수천 코어까지 확장

사용자 인터페이스

GUI(graphical user interface) 없음 — 텍스트 명령 한 줄씩으로 계 정의 · 실행 (`in` · `data` · `include`)

확장 방식

기능은 패키지(package) 단위로 추가 — 빌드 때 포함한 패키지의 명령만 사용 가능

이 강좌가 쓰는 범위 — 유기 분자 벌크와 계면 · GAFF2(General AMBER Force Field 2) 계열 힘장 · NVT/NPT 앙상블(ensemble) · 밀도 · CED(cohesive energy density) · 점도 · 부착일(work of adhesion)

Ubuntu 에서 LAMMPS 설치 — 준비와 CPU 빌드

00 먼저 결정할 것

CPU 전용인가 GPU 를 쓸 것인가 · 필요한 패키지(MOLECULE · KSPACE · RIGID · EXTRA-DUMP) · 정밀도(double 기본) · 버전은 stable 브랜치 고정

GPU 가 없거나 계가 10만 원자 이하면 CPU + MPI 만으로 충분 — 빌드 단순 · 스타일 제약 없음

01 의존성 설치 (apt)

```
sudo apt update
sudo apt install -y build-essential cmake git \
  python3-dev libopenmpi-dev openmpi-bin \
  libfftw3-dev libjpeg-dev libpng-dev
```

MPI · FFTW3(고속 푸리에 변환 라이브러리)는 병렬과 kspace(pppm, particle–particle particle–mesh)에 필수 · jpeg · png 는 dump image 용

02 소스 받기 · 빌드 디렉터리

```
git clone -b stable https://github.com/lammps/lammps.git
cd lammps && mkdir build && cd build
```

03 CPU 빌드 (cmake)

```
cmake -D BUILD_MPI=yes -D BUILD_OMP=yes \
  -D PKG_MOLECULE=yes -D PKG_KSPACE=yes \
  -D PKG_RIGID=yes -D PKG_EXTRA-DUMP=yes \
  -D FFT=FFTW3 -D CMAKE_BUILD_TYPE=Release ../cmake
make -j$(nproc)
sudo make install   # lmp 실행 파일 설치
```

확인 — `lmp -h | head` 로 컴파일된 패키지 목록 점검

conda 는 어디에 필요한가

LAMMPS 본체는 apt + 소스 빌드가 안정적 · conda 는 주변 파이썬 도구(RDKit · OpenBabel · MDAnalysis · moltemplate · packmol · pandas)를 격리하는 용도

주의 — conda 환경의 openmpi · gcc 가 시스템 것과 섞이면 실행 시 심볼 오류 · LAMMPS 를 돌릴 때는 conda deactivate 또는 MPI 없는 별도 env 사용

동작 확인

```
cd ../bench
mpirun -np 4 lmp -in in.lj   # 3d LJ melt, 정상 종료 확인
```

GPU 빌드 — 드라이버 · 아키텍처 · KOKKOS

04 드라이버 · CUDA 확인

```
nvidia-smi              # Driver Version / CUDA Version
nvcc --version          # toolkit 설치 여부
sudo apt install -y nvidia-cuda-toolkit   # 없을 때
```

드라이버가 CUDA(Compute Unified Device Architecture) 버전을 결정 — 드라이버 ≥ 550 이면 CUDA 12.x · 드라이버 먼저 갱신 후 toolkit 맞춤

05 GPU 아키텍처 인자

RTX 3090	Kokkos_ARCH_AMPERE86
A100	Kokkos_ARCH_AMPERE80
RTX 4090	Kokkos_ARCH_ADA89
H100 · H200	Kokkos_ARCH_HOPPER90

06 KOKKOS 빌드 (권장)

```
cmake -D BUILD_MPI=yes -D PKG_KOKKOS=yes \
  -D Kokkos_ENABLE_CUDA=yes -D Kokkos_ARCH_HOPPER90=on \
  -D PKG_MOLECULE=yes -D PKG_KSPACE=yes -D PKG_RIGID=yes \
  -D FFT=FFTW3 -D CMAKE_BUILD_TYPE=Release ../cmake
make -j$(nproc)
```

실행 방법 · 정밀도 · 검증

07 실행 방법

```
# GPU 1장 · MPI 2랭크
mpirun -np 2 lmp -k on g 1 -sf kk \
  -pk kokkos newton on neigh half -in in.run
```

MPI 랭크(rank) 수는 GPU 장수의 1 ~ 4배 · 랭크 과다 시 GPU 경합으로 속도 저하

정밀도 선택

KOKKOS 는 기본 double · GPU 패키지는 `-D GPU_PREC=mixed|double`

mixed 는 1.5 ~ 2배 빠르지만 장시간 NVE 에너지 드리프트(energy drift)가 커질 수 있음 → 물성 생산 런은 double, 선별 런은 mixed

주의 · 검증

KOKKOS 가 지원하지 않는 pair · fix 를 쓰면 그 부분만 CPU 로 폴백(fallback) → 성능이 오히려 하락 · 로그의 "(kk)" 표시로 확인

같은 입력을 CPU 와 GPU 로 각 1000 스텝 돌려 초기 에너지 · 압력이 유효숫자 5자리까지 일치하는지 대조

실험 ID 에 빌드 종류(cpu · kk-double · kk-mixed)를 기록 — 정밀도가 다른 결과의 한 데이터셋 혼합 금지

4-1 · PACKAGES

패키지 구조와 두 가지 가속 경로

package	추가 기능	이 강좌
MOLECULE	결합 · 각 · 이면각 항	필수
KSPACE	장거리 Coulomb (pppm · ewald)	필수
RIGID	강체 · 구속 물 모델	선택
GPU	pair · kspace 일부만 GPU 로	권장
KOKKOS	대부분의 커널을 GPU · 스레드로	대형 계

GPU 패키지

원자 데이터는 CPU 에 두고 짧은 거리 상호작용만 GPU 로 전달 · 중소형 계에서 도입 용이

```
mpirun -np 8 lmp -sf gpu -pk gpu 1 -in in.binderA
```

KOKKOS 패키지

원자 데이터를 GPU 메모리에 상주시켜 전송 감소 · 수십만 원자 이상에서 유리 · 지원 스타일 사전 확인

```
lmp -k on g 1 -sf kk -pk kokkos newton on -in in.binderA
```

가속 패키지 = 결과 불변 — 같은 입력에 같은 물리, 속도만 차이 (부동소수 정밀도 차이는 별도)

LAMMPS 빌드를 Claude Code 에 맡길 때의 프롬프트

① LAMMPS 빌드 요청

```
Ubuntu 22.04 · CUDA 12.x 장비에 LAMMPS 를 설치하는
절차를 실행 가능한 셸 스크립트로 작성해 줘.

[버전]   stable 브랜치 · 커밋 해시까지 고정
[빌드]   cmake · MPI 병렬 · 빌드 디렉터리 분리
[패키지] MOLECULE KSPACE RIGID EXTRA-DUMP 필수
         GPU(KOKKOS) 는 선택 분기로
[환경]   conda 환경 comid (python 3.11) 기준
[검증]   lmp -h 로 설치 패키지 목록 확인 후
         예제 bench/in.lj 를 4코어로 돌려 성공 판정

제약 — sudo 가 필요한 줄은 따로 표시. 드라이버 ·
CUDA · 아키텍처(sm_XX) 매칭 확인 단계를 넣고,
GPU 미지원 스타일일 때의 CPU 폴백도 적어라.
```

두 프롬프트의 공통 조건

버전 고정 · 검증 명령 · 실패 시 분기 — 셋 중 하나라도 없으면 "설치됨"만 확인하고 종료

받은 뒤 반드시 할 일

lmp 버전 · 패키지 목록 · packmol 버전을 meta.json 에 기록 — 이후 결과 차이의 원인 추적용

Packmol · 빌더 설치를 맡길 때의 프롬프트

② Packmol · 빌더 요청

```
Packmol 을 설치하고 조성 명세에서 LAMMPS data
파일까지 만드는 파이프라인을 작성해 줘.

[설치] conda-forge packmol · rdkit · jinja2 · pyyaml
       버전을 고정해 environment.yml 로
[입력] spec.yaml — 성분 wt% · 분자량 · 목표 원자 수
[처리] wt% → 분자 개수 환산 → RDKit 3D 임베딩 →
       pack.inp 생성 → packmol 실행 → pdb → data
[검증] 원자 수 · 총전하 0 · 최근접 거리 하한 ·
       목표 조성과의 wt% 편차 ±0.5 이내

제약 — 초기 밀도는 목표의 절반으로 잡고 나머지는
NPT 압축에 맡겨라. tolerance 실패 시 상자를 5%
키워 재시도하는 루프를 넣어라.
```

맡기지 않는 것

설치 자체는 위임 가능 · 필요한 패키지 판단은 사용할 명령 기준으로 사람이 수행

받은 코드에서 고칠 값

「목표의 절반」 초기 밀도는 고리 관통이 잦은 구간 — 4-4 품질 점검의 실측대로 0.2 g/cm³ 부근으로 낮추고, 재시도는 상자 확대 대신 시드 변경으로

4-2

입력 스크립트 해부

16분 · units · atom_style · pair_style · fix · run

4-2 · FILE MAP

LAMMPS 계산의 파일 구성



보라 상자 · 실행 엔진

파일이 아니라 프로그램 — 입력 세 종류를 읽어 궤적과 로그를 구성

data 파일에 들어가는 것

원자 수 · 상자 크기 · Masses · Atoms(전하 포함) · Bonds · Angles · Dihedrals

in 스크립트가 정하는 것

단위계 · 비결합 설정 · 앙상블 · 시간 간격 · 출력 주기 · 실행 길이

출력이 쓰이는 곳

log → 평형 판정 · 궤적(trajectory) → 5~7주차 물성 · final.data → 다음 조건 계산

확장자 규약 LAMMPS는 확장자 강제 없음 · 입력 스크립트는 `in.binderA` · `binderA.in` · `binderA.lmp` 모두 사용 · 구조 파일도 `.data` · `data.binderA` · 도구에 따라 `.lmp`로 산출 — 강좌에서는 in.* / *.data 로 통일

in.binderA — 실행 절차 파일

```
units       real
atom_style  full
boundary    p p p

read_data   binderA.data

pair_style    lj/cut/coul/long 12.0
kspace_style  pppm 1.0e-4
special_bonds amber

velocity    all create 298 4928459
timestep    1.0
fix   npt   all npt temp 298 298 100 &
            iso 1.0 1.0 1000

thermo_style custom step temp press density
dump   d1   all custom 5000 prod.lammpstrj
run         1000000
```

초기화	단위계 · 원자 스타일 · 경계조건 — 나중에 바꿀 수 없음
원자 정의	read_data로 3주차 셀 읽기 — 질량 · 전하 · 결합 정보 포함
힘장 설정	pair_style · kspace · special_bonds — 3주차에서 고른 힘장과 일치 필수
실행 설정	초기 속도 · 앙상블 fix · 시간 간격
출력 · 실행	thermo · dump 주기와 run — 출력 주기가 곧 표본 간격

명령은 위에서 아래로 순차 해석 · 같은 명령 재사용 시 앞의 설정 덮어씀

스크립트의 다섯 구간 — 각 구간이 정하는 것



상자 · 단위 먼저 결정
단위 · 원자 스타일 · 경계조건

상자에 원자 배치
질량 · 전하 · 결합 목록

원자 간 힘 결정
컷오프 · 정확도 · 1-4 축소

운동 시작 · 제어
초기 속도 · 앙상블 fix

기록 항목 · 주기 결정
출력 주기 = 표본 간격

앞 장 코드의 색 띠 = 이 다섯 칸의 색 · 명령의 소속 구간부터 확인하면 낯선 스크립트도 판독 가능

4-2 · UNITS

단위계와 data 파일

물리량	units real	units metal
거리	Å	Å
에너지	kcal/mol	eV
시간	fs	ps
압력	atm	bar

binderA.data

```
  98304 atoms      68  atom types
  99840 bonds      142 bond types

  0.0 105.82  xlo xhi
  0.0 105.82  ylo yhi
  0.0 105.82  zlo zhi

Masses / Pair Coeffs / Bond Coeffs
Atoms  # full : id mol type q x y z
Bonds / Angles / Dihedrals
```

원자 수 · 상자 크기 · 전하 합이 0인지 먼저 확인

단위계는 스크립트 첫 줄에서 한 번만 결정 — 이후 모든 수치의 의미가 여기에 연동

4-2 · PAIR STYLE

힘장에 대응하는 pair_style

힘장	pair_style	special_bonds
GAFF2 · AMBER	`lj/cut/coul/long 12.0`	`amber`
OPLS-AA	`lj/cut/coul/long 12.0 (geometric)`	`lj/coul 0 0 0.5`
COMPASS · PCFF	`lj/class2/coul/long 12.0`	`lj/coul 0 0 1`
ClayFF (계면)	`lj/cut/coul/long + hybrid/overlay`	유기 쪽 규약 적용

`pair_modify` mix arithmetic · tail yes 로 조합 규칙(combining rule)과 꼬리 보정(tail correction)을 명시 — 기본값에 의존하지 않음

장거리 Coulomb과 계산 비용

pppm 1.0e-4

상대 힘 오차 기준값 · 벌크 계산의 표준 설정

컷오프 10 → 14 Å

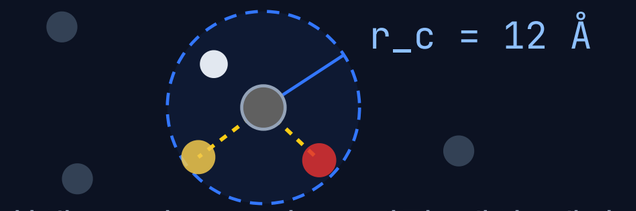
실공간(real space) 비용 증가 · 역공간(reciprocal space) 격자는 성김 — 총 비용 최소점 존재

slab 보정

계면 계처럼 한 축이 비주기면 kspace_modify slab 3.0 지정

```
kspace_style   pppm 1.0e-4
kspace_modify  mesh 60 60 60 order 5
neighbor       2.0 bin
neigh_modify   every 1 delay 0 check yes
```

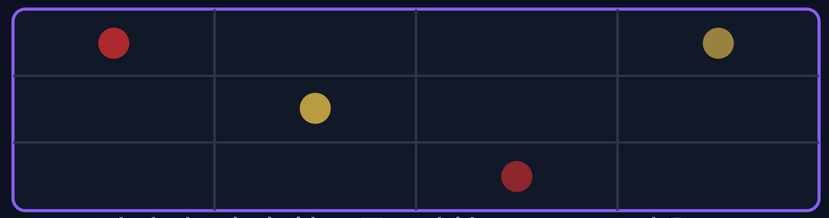
실공간 · 컷오프 안쪽만 직접 합산



회색 = 컷오프 밖 · 직접 계산 제외

나머지

역공간 · 격자에 전하 할당 후 FFT로 합산 (pppm)



격자가 촘촘할수록 정확도 ↑ · 비용 ↑

$$U_{coul} = U_{real} + U_{kspace}$$

컷오프 · 정확도 설정은 데이터셋 전체에서 고정 — 값이 다르면 물성 비교가 성립하지 않음

4-2 · COMPLETE EXAMPLE squalane 100 + DOCHN 100 · 298 K · 밀도

두 성분 셀의 밀도를 구하는 전체 스크립트

```
① units         real
   atom_style   full
   boundary     p p p
② read_data mix2.data
③ pair_style · kspace · special_bonds
④ minimize
⑤ velocity · timestep · fix npt · run
⑥ variable · ave/time · dump · run
⑦ write_data · write_restart
```

셀 상태

ρ = — g/cm³



상자와 단위만 정해진 상태 — 원자 없음

◀ ▶ 1 / 7 단락 클릭 → 해당 줄 확대 · 그림 갱신

① 초기화 — units · atom_style · boundary

units real
Å · kcal/mol · fs · g/cm³

atom_style full
분자 ID · 전하 · 결합 보유

boundary p p p
세 방향 주기 경계 · 벌크

주의 — 이 세 줄은 이후 변경 불가 · data 파일의 열 구성과 힘장 계수 해석이 모두 이 설정에 의존

산출물 없음 · 이후 모든 명령의 해석 기준

명령 한 줄씩 — 초기화부터 평형까지

명령	인수의 의미	빠지거나 틀리면
`units real`	Å · kcal/mol · fs · g/cm³ 로 모든 수치를 해석	힘장 계수가 통째로 오해석
`atom_style full`	원자마다 분자 ID · 전하 · 결합 보유	data 파일의 열 수가 안 맞아 읽기 실패
`boundary p p p`	x · y · z 모두 주기 — 벌크 의미	f 로 두면 표면 생성 → 밀도 하락
`read_data mix2.data`	상자 크기 · 원자 · 전하 · 결합 목록 읽기	원자 0개 — 계산 대상 없음
`pair_style … 12.0`	LJ + 실공간 쿨롱 · 컷오프 12 Å	컷오프를 줄이면 응집이 약해져 밀도 하락
`pair_modify mix · tail`	이종쌍(unlike pair) σ 를 산술평균 · 컷오프 밖 꼬리 에너지 보정	tail no 면 밀도 1~2% 과소 산출
`kspace_style pppm 1e-4`	장거리 쿨롱을 역공간에서 · 상대 오차 10^{-4}	없으면 전하 있는 계의 에너지가 발산
`special_bonds amber`	1-2 · 1-3 비결합 제외 · 1-4 는 쿨롱 0.8333 · LJ 0.5 배	결합된 원자끼리 두 번 세어 밀도 과대
`minimize 1e-4 1e-6 5000 50000`	에너지 · 힘 허용오차 · 최대 반복 · 최대 힘 평가 횟수	겹친 원자가 남아 첫 스텝에서 발산
`velocity all create 298 4928459`	298 K 맥스웰(Maxwell–Boltzmann) 분포로 초기 속도 · 뒤 숫자는 난수 시드(random seed)	시드를 고정하지 않으면 재현 불가
`timestep 1.0`	1 fs — C–H 진동 주기의 1/10 수준	2 fs 이상이면 에너지 표류
`fix npt … 100 iso 1 1 1000`	목표 온도 시작·끝과 온도 완화시간 100 fs · 압력 1 atm 과 완화 1000 fs · iso 는 세 축 같은 비율	완화시간이 짧으면 T · P 진동

출력과 마무리 — 항목별 저장 파일

명령	인수의 의미	산출물
`thermo_style custom …`	로그에 쓸 항목 — step · temp · press · density	log 파일 시계열
`thermo 1000`	1000스텝(1 ps)마다 한 줄 출력	수렴 점검용 시계열
`run 500000`	1 fs × 500,000 = 500 ps 진행	평형화 구간
`variable dens equal density`	호출 시점의 밀도를 값으로 주는 변수 정의	평균 계산의 입력
`fix ave/time 100 50 5000`	100스텝마다 표본을 뜨고 50개를 평균해 5000스텝마다 한 줄 기록	density.txt · 200개 값
`dump d1 all custom 5000 …`	5000스텝마다 id · type · 좌표를 저장	prod.lammpstrj
`write_data final.data`	현재 상자 · 좌표 · 전하 · 결합을 **텍스트**로 저장 — read_data 로 다시 읽기	다음 계산의 입력 구조
`write_restart final.restart`	좌표에 더해 **속도 · fix 내부 상태 · 스텝 수**까지 이진으로 저장 — read_restart 로 계속 실행	중단 복구 지점

`final.data` 를 쓸 때
조건을 바꿔 새로 시작할 때 — 온도를 바꾸거나 계면을 붙일 때의 출발 구조

`final.restart` 를 쓸 때
같은 계산을 끊김 없이 계속 실행할 때 — 속도와 써모스탯(thermostat) 상태 보존 → 통계 연속

`restart` 의 제약
이진 파일이라 사람이 판독 불가 · LAMMPS 버전 변경 시 열리지 않을 수 있음 — 두 파일 모두 보관이 안전

4-2 · WHAT THE RUN DOES

구간별 셀의 변화

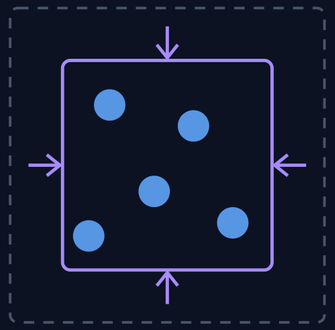
④ 최소화



겹친 원자 분리

상자 크기는 그대로

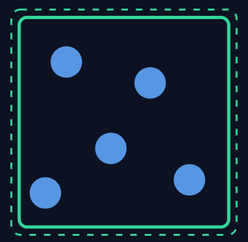
⑤ NPT 압축 평형



상자 축소 · 밀도 상승

이 구간은 폐기

⑥ 생산 구간



크기는 한 값 주위로 요동

5000스텝마다 한 값 기록

밀도 평균



density.txt 후반 값 평균

블록 표준편차가 오차

밀도 산출 방법

$M = 100 \times 422.8 + 100 \times 406.7$

$= 82{,}950\ \mathrm{g/mol}$

$\rho = M / (N_A \cdot \langle V \rangle)$

예상 결과

$\langle \rho \rangle \approx 0.89\ \mathrm{g/cm^3}$ · 한 변 약 54 Å · density.txt 에 200개 값 누적

확인 순서

log 의 온도 · 압력이 설정값 주위 → 밀도 시계열 평탄 → 전·후반 평균 차이 0.5% 이내

4-2 · OUTPUT FILES

출력 파일 — log · 궤적 · 구조 · restart

log.binderA · prod.lammpstrj

```
Step    Temp    Press   Density
     0  298.0  1842.5   0.8123
100000 297.8    18.4    0.9781
500000 298.1     2.1    0.9968
```

```
ITEM: TIMESTEP
500000
ITEM: NUMBER OF ATOMS
98304
ITEM: ATOMS id type x y z
```

log.*	평형 판정 · 밀도 평균 · 경고 확인
*.lammpstrj	RDF(radial distribution function) · MSD(mean squared displacement) · 점도 — 5~7주차 물성 계산의 원본
final.data	평형 구조 — 다른 온도 · 계면 계산의 입력으로 재사용
*.restart	중단 복구 — 이진 형식이라 버전이 바뀌면 못 읽을 수 있음

덤프 주기가 표본 간격 — 너무 촘촘하면 용량만 커지고 정보는 늘지 않음

출력 네 종류는 실험 ID 폴더 하나에 함께 보관 — 8주차 데이터 적재는 이 폴더 구조를 그대로 판독

4-2 · RUN PROTOCOL

네 단계 실행 프로토콜

단계	설정	목적 · 판정
01 최소화	minimize 1e-4 1e-6	겹침 제거 · 최대 힘이 유한값으로 수렴
02 어닐링	NPT 600 → 298 K · 1 ns	초기 배치 의존성 제거(어닐링, annealing) · 분해 온도 아래 유지
03 평형화	NPT 298 K · 2 ns	밀도 · 부피 추세 소멸 — 2주차 판정 기준 적용
04 생산	NPT · NVT 2 ~ 10 ns	물성 샘플링 · 이 구간만 통계에 사용



최소화 권장 조건
겹침이 심하면 soft 포텐셜로 먼저 밀어낸 뒤 minimize

어닐링 권장 조건
분해 온도 아래 (바인더 ≤ 600 K) · 냉각률 ≤ 0.5 K/ps

평형화 권장 조건
NPT · 밀도 변화 < 0.5%/ns 이면 통과

생산 권장 조건
밀도·CED 2 ns · 점도 10 ns 이상 · 시드 3개

단계 조합 네 단계는 고정 순서가 아니라 목적에 맞춰 조합 — 이미 평형화된 구조를 다른 온도로 옮길 때는 어닐링 생략, 점도는 NPT 평형 후 NVT 생산, 계면 계는 평형화를 aniso로 한 단계 더, Tg는 어닐링을 여러 냉각률로 반복

4-2 · PROMPT FOR THE AGENT 그대로 복사해 쓰는 요청문

이 스크립트를 Claude Code 에 요청하는 프롬프트

prompt.txt

```
아스팔트 바인더 벌크 셀의 밀도를 구하는 LAMMPS
실행 한 벌을 만들어 줘. 조건은 다음과 같다.

[계]   squalane 100 + DOCHN 100 · 목표 약 1.7만 원자
[힘장] GAFF2 · lj/cut/coul/long 12.0 · pppm 1.0e-4
       mix arithmetic · tail yes · special_bonds amber
[절차] minimize → NPT 298 K 500 ps → NPT 생산 1 ns
[측정] 밀도 · 5000스텝마다 density.txt 에 평균 기록
[산출] in.binderA · mix2.data 를 만드는 파이썬 빌더
       run.sh · final.data · final.restart 저장까지

제약 — units real / atom_style full 은 고정. 온도 ·
시드 · 스텝 수를 스크립트에 직접 적지 말고 spec.yaml
에서 읽어 Jinja2 템플릿으로 렌더해라.

검증 — 생성 직후 타입 수 = Masses 행 수, 총전하 0,
모든 타입 쌍의 pair_coeff 정의를 점검하는 함수와
1000스텝 시험 실행 명령을 함께 내놓아라.

설명은 짧게, 파일별 전체 내용을 코드로 제시해라.
```

채워야 하는 다섯 칸

계 · 힘장 · 절차 · 산출물 · 검증 — 하나라도 비면 에이전트가 임의값으로 채움 → 결과 변동

나쁜 요청의 예

"LAMMPS 밀도 계산 스크립트 만들어 줘" — 단위계 · 컷오프 · 평형 길이 누락 → 그럴듯하지만 재현 불가능한 입력 산출

받은 결과를 보는 순서

① 앞의 일곱 단락 모두 존재 → ② 하드코딩(hard-coding)된 숫자 잔존 여부 → ③ 1000스텝 시험 통과

사람이 지켜야 할 것

힘장 선택과 평형 길이의 타당성은 위임 금지 — 코드 작성은 에이전트, 물리적 판단은 사람 담당

4-3

데이터 파일과 토폴로지

8분 · 원자 좌표 · 토폴로지 · 힘장 파라미터

data 파일 — 구조와 토폴로지

ch4.data · 예시

```
5 atoms       2 atom types
4 bonds       1 bond types

0.0 78.42  xlo xhi
0.0 78.42  ylo yhi
0.0 78.42  zlo zhi

Masses
  1  12.011   # c3
  2   1.008   # hc

Atoms  # full
  1  1  1  -0.108   0.00  0.00  0.00
  2  1  2   0.027   0.63  0.63  0.63
  3  1  2   0.027  -0.63 -0.63  0.63

Bonds
  1  1  1  2
  2  1  1  3
```

섹션 순서

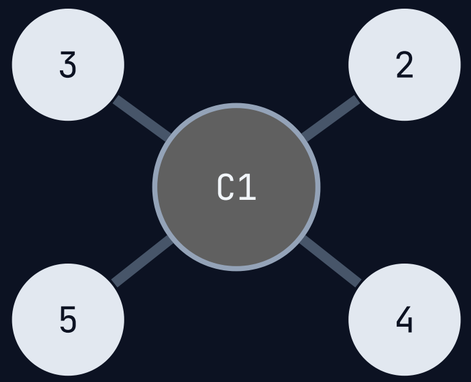
개수 → 상자 → Masses → Atoms → Bonds · 왼쪽 색 띠와 같은 순서

atom type · bond type 번호 자체는 임의 지정 가능 · 의미는 그 번호에 붙는 계수가 결정

bond type은 같은 종류의 결합끼리 묶는 번호 — C–H와 C–C는 다른 type이어야 서로 다른 `bond_coeff`(K, r_0) 적용

data 파일 읽기 — 메탄 한 분자의 번호와 연결

예시 · 메탄 CH_4 한 분자



Box bounds — 0.0 – 78.42 Å · x · y · z 주기 경계

Masses — type · 질량 · 원자타입

type	mass	label
1	12.011	c3 (C)
2	1.008	hc (H)

Atoms — 원자 번호가 그림의 번호

id	mol	type	q	x	y	z
1	1	1	-0.108	0.00	0.00	0.00
2	1	2	0.027	0.63	0.63	0.63
3	1	2	0.027	-0.63	-0.63	0.63

Bonds — 연결된 두 원자

id	type	atom 1	atom 2
1	1	1	2
2	1	1	3

여기 예시는 메탄 한 분자 · 실제 binderA.data 는 같은 형식이 98304 atoms · 68 atom types 로 확장된 것 — Pair Coeffs · Bond Coeffs 는 include 파일로 분리

ff_gaff2.inc — 힘장 계수 파일

ff_gaff2.inc

```
pair_style    lj/cut/coul/long 12.0
pair_modify   mix arithmetic tail yes
kspace_style  pppm 1.0e-4
pair_coeff    1 1  0.0860  3.3997

bond_style    harmonic
bond_coeff    1  478.4  1.387

angle_style   harmonic
angle_coeff   1  63.1  120.0

special_bonds amber

dihedral_style fourier
```

pair	$U = 4\varepsilon[(\sigma/r)^{12} - (\sigma/r)^{6}]$ · ε = 0.0860 kcal/mol · σ = 3.3997 Å (타입 1 = 방향족 탄소)
bond	$U = K(r - r_0)^2$ · K = 478.4 kcal/mol·Å² · r_0 = 1.387 Å
angle	$U = K(\theta - \theta_0)^2$ · K = 63.1 kcal/mol·rad² · θ_0 = 120°
special_bonds	1-2 = 0 · 1-3 = 0 · 1-4 : vdW 0.5 / Coulomb 0.8333
dihedral	fourier 형식 · 이면각 계수는 타입별로 별도 지정 — 이 예시에서는 생략

계수의 원자 타입 번호는 data 파일의 타입 번호와 정확히 대응 필수 — 어긋나면 경고 없이 틀린 결과 산출

4-4

초기 구조 생성

10분 · Packmol · Moltemplate · RDKit · 벌크 셀 구축과 품질 점검

초기 구조를 만드는 세 가지 도구

RDKit

단분자 3D 좌표

SMILES(Simplified Molecular Input Line Entry System) → 3D 임베딩(embedding) → 간이 최적화 · 분자 하나의 pdb · mol 파일 생성

`AllChem.EmbedMolecule · MMFFOptimize`

Packmol

상자 채우기

여러 분자를 겹치지 않게 목표 개수 · 목표 밀도로 배치 · 영역(region)별 배치도 가능

`structure · number · inside box`

Moltemplate

토폴로지까지 포함한 data

분자 정의와 힘장 계수를 함께 기술해 LAMMPS data 파일을 직접 생성 · 반복 단위 · 격자 배치에 유리

`moltemplate.sh system.lt`

실제 흐름 — RDKit 로 단분자 좌표 → Packmol 로 목표 조성만큼 채움 → 힘장 타이핑(antechamber · Moltemplate) → `binderA.data`

좌표만 만드는 도구와 토폴로지까지 만드는 도구의 구분 필수 — 결합 정보의 생성 위치 추적용

Packmol — 역할과 설치

역할

여러 종류의 분자를 정해진 개수만큼 상자 안에 겹치지 않게 배치해 MD 초기 좌표 생성 — 최소 원자간 거리를 제약으로 둔 최적화

SARA(Saturates · Aromatics · Resins · Asphaltenes) 13종 혼합 바인더처럼 조성비가 정해진 계를 만들 때 사실상 표준 도구

설치 A · conda (간편)

```
conda create -n md python=3.11
conda activate md
conda install -c conda-forge packmol openbabel rdkit
```

설치 B · 소스 (최신판)

```
sudo apt install -y gfortran
git clone https://github.com/m3g/packmol.git
cd packmol && make
sudo cp packmol /usr/local/bin/
```

동작 확인

```
packmol < example.inp      # 표준입력으로 받는다
# 종료 메시지에 "Success!" 가 나와야 정상
```

Packmol 은 좌표만 생성 — 결합 · 원자 타입 · 전하는 다음 단계(3주차 파라미터화, parameterization)에서 부여

입력 준비물

분자별 단일 분자 좌표 파일(pdb · xyz) — RDKit 으로 SMILES → 3D 생성 후 저장 · 재현이 필요하면 임베딩 함수에 시드 인자를 지정

```
from rdkit import Chem
from rdkit.Chem import AllChem
m = Chem.AddHs(Chem.MolFromSmiles("Cc1ccccc1"))
AllChem.EmbedMolecule(m)
AllChem.MMFFOptimizeMolecule(m)
Chem.MolToPDBFile(m, "toluene.pdb")
```

입력 파일 작성 — 개수 계산과 inp 문법

binder.inp — 아스팔트 바인더 예시

```
tolerance 3.0
filetype pdb
output binder.pdb
seed 20240901

structure asphaltene.pdb
  number 12
  inside box 2. 2. 2. 78. 78. 78.
end structure

structure resin.pdb
  number 28
  inside box 2. 2. 2. 78. 78. 78.
end structure

structure saturate.pdb
  number 40
  inside box 2. 2. 2. 78. 78. 78.
end structure
```

개수는 목표 밀도에서 역산

$$N_i = x_i \cdot \rho \, V \, N_A / \bar{M}$$

x_i 몰분율(mole fraction) · ρ 목표 밀도 1.0 g/cm³ · V 상자 부피 · $\bar{M}$ 평균 분자량 · N_A 아보가드로 수

예 — 80 Å 상자(5.12e−19 cm³) · ρ 1.0 · $\bar{M}$ 480 g/mol → 총 약 640 분자 · 초기 0.2 g/cm³ 상자는 한 변 약 137 Å

실무 팁

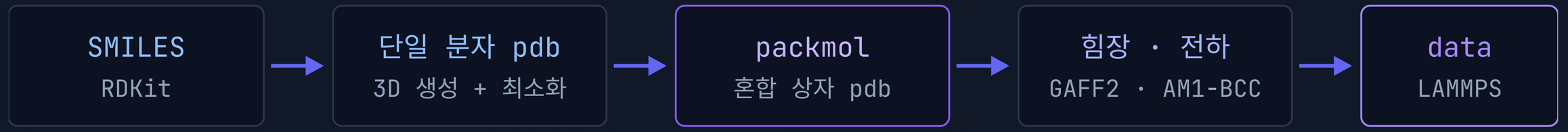
tolerance 3.0 Å 권장(4-4 품질 게이트 실측) — 너무 작으면 원자 접근 · 고리 관통으로 최소화 발산, 너무 크면 패킹 실패

상자는 초기 밀도 0.2 g/cm³ 기준으로 확대 → NPT 압축으로 목표 밀도 수렴

주기경계를 쓸 것이므로 상자 경계에서 2 Å 안쪽(inside box 2. … 78.)에 배치

seed 를 기록 — 같은 조성이라도 초기 배치가 달라 시드가 결과의 일부

Packmol 결과를 LAMMPS data 로 연결



좌표 = packmol · 결합 · 타입 · 전하 = 파라미터화 단계 — 두 결과 병합 → data 파일

moltemplate

분자 템플릿(.lt)에 힘장을 기술하고 packmol 좌표를 병합해 data 를 출력 · 이 강좌의 기본 경로

AmberTools

antechamber 로 GAFF2 타입 · AM1-BCC 전하 → tleap → parmed 로 LAMMPS data 변환

InterMol · MDAnalysis

포맷 변환과 구조 점검 보조 · 원자 수 · 전하 합 확인 스크립트 작성에 사용

data 파일을 돌리기 전 검증 체크리스트

01	원자 수 = Σ (분자 수 × 분자당 원자 수) 와 일치	헤더의 atoms 값 대조
02	총 전하 합이 0 (중성계) · 분자별 전하 합도 0	|Σq| < 1e-4 e
03	최소 원자간 거리 확인 — 너무 가까우면 최소화가 발산	r_min > 1.8 Å
04	발산 · 관통 시 시드 변경(change_seed) 후 재패킹 · 밀도 · tolerance 유지	0.2 g/cm³ · 3.0 Å
05	주기경계 겹침 — 경계 여유가 없으면 첫 스텝에서 압력 급증	경계 안쪽 2 Å
06	packmol 좌표는 무작위 배치 — 반드시 어닐링 후 물성 계산	4-2 프로토콜

4-4 · PACKING

벌크 셀 구축 절차

01 목표 조성에서 분자별 개수 결정 — 총 원자 수 5만~10만이 실용적 범위

02 초기 밀도 0.2 g/cm³(목표의 약 1/5)의 큰 상자에 무작위 배치 — 겹침 · 고리 관통 방지

03 에너지 최소화 후 짧은 NVT로 국소 구조를 완화

04 NPT로 압축해 평형 밀도 도달 — 4-2의 네 단계 실행 프로토콜로 연결

계가 작으면 통계 오차 증가, 크면 GPU 시간 증가 · 물성별 최소 크기를 미리 설정

4-4 · PACKING QUALITY

Packmol 출력은 검사 통과 후 사용

실제로 일어난 일 · 플랫폼 벌크 실험

96 / 165 건이 고리 관통(ring threading)으로 오염

한 분자의 결합이 다른 분자의 방향족 고리를 꿰뚫은 채 패킹 — 관통은 위상 불변량(topological invariant)이라 최소화 · MD로 해소 불가

압축 시 결합 비정상 신장 — 방향족 C–C 2.67 Å (정상 1.40 Å) · CED 음수 값이 나온 뒤에야 발견

초기 밀도 · tolerance와 첫 시도 청정률

초기 밀도	tolerance	청정률
0.5 g/cm³	2.0 Å	25%
0.2 g/cm³	2.0 Å	58%
0.2 g/cm³	3.0 Å	92%

"목표의 절반"(≈ 0.5)은 최악 구간 · 밀도가 낮고 tolerance가 클수록 관통 감소

게이트(gate) — data 파일 변환 전 선별

0 Packmol 자체 수렴 보고 확인 — Success! · 위반량 0 · exit 0이 아니면 실패

1 분자간 최소 원자거리 검사 — 분자 내 결합은 제외하고 계수

2 실패하면 시드를 바꿔(change_seed) 재패킹 · 상한을 넘으면 중단

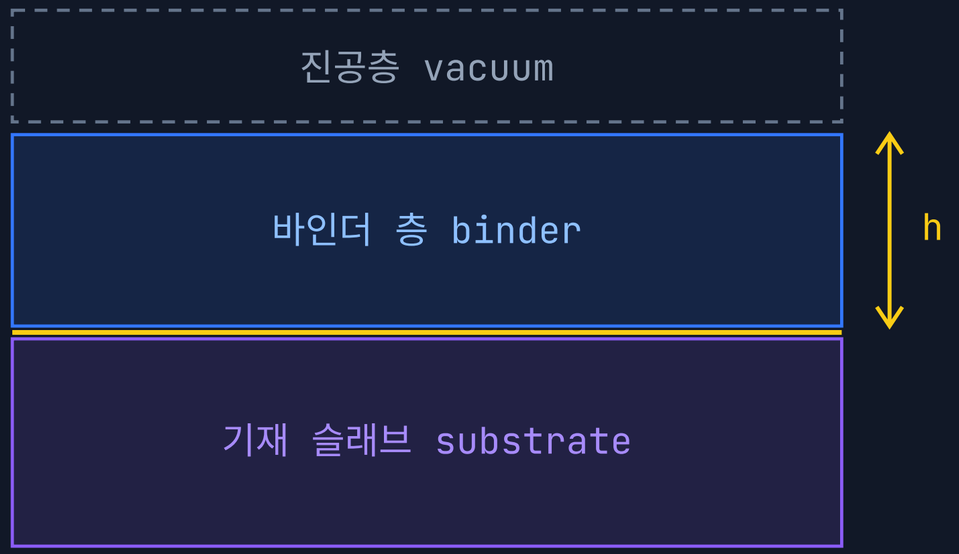
스윕 권장점 ρ = 0.2 g/cm³ · tolerance 3.0 Å · maxit 200 — 첫 시도 92%, 3회 재시도로 약 99.9%

4 - 5

다층구조 생성

8분 · 부착 · 계면 연구를 위한 다층 시스템

부착 · 계면 연구를 위한 다층 셀



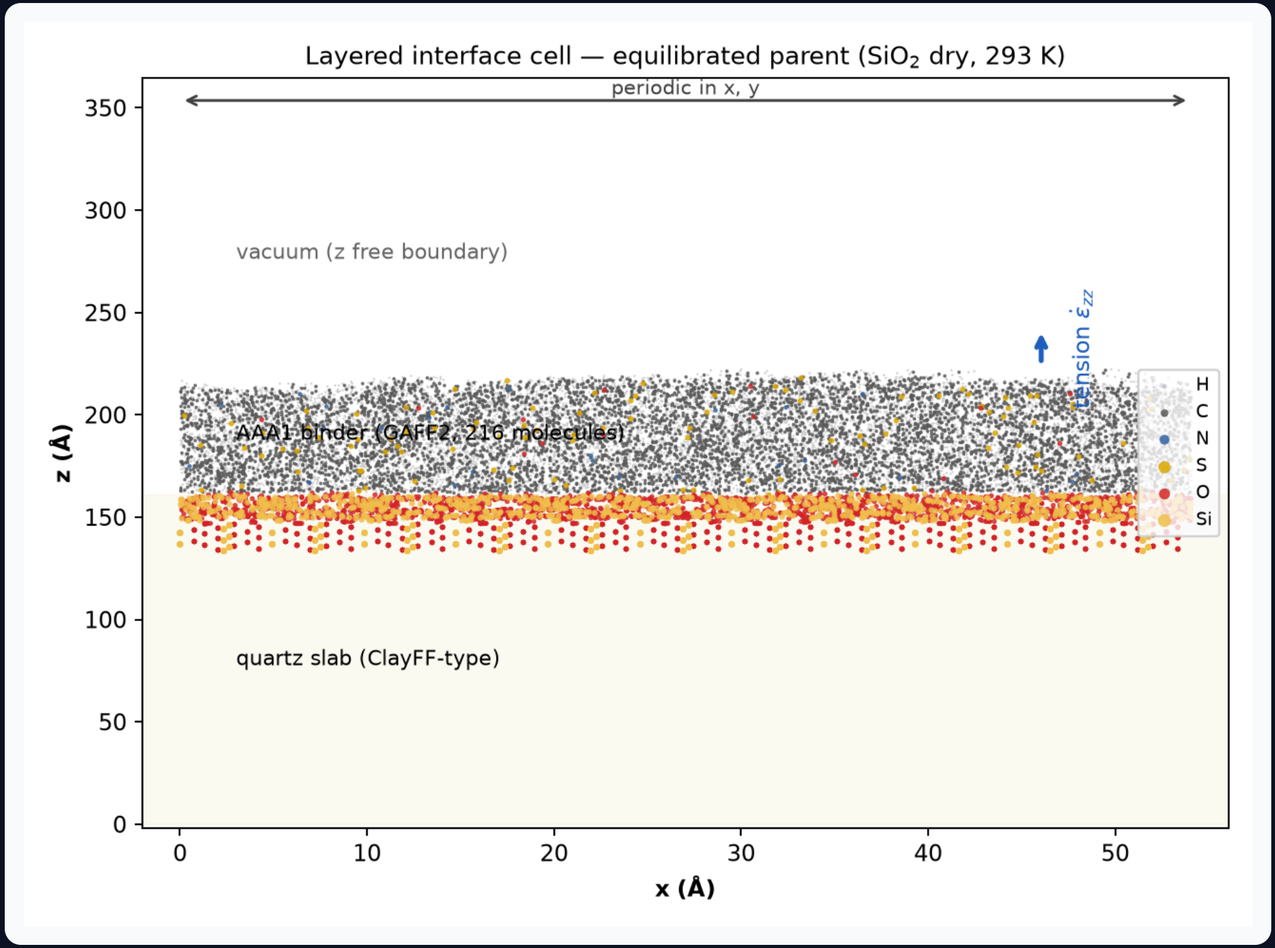
z ↑ · x · y 주기 p p · z 는 f (고정)

슬래브	골재 · 시멘트 상 결정 셀을 반복해 만든 고체 층 — 아래 몇 층은 위치 고정(fix setforce 0)
바인더 층	4-2 프로토콜로 미리 평형화한 벌크 블록을 슬래브 위에 배치
진공층	주기 이미지끼리 닿지 않도록 z 방향에 최소 30 Å 이상 — slab 보정과 함께 사용
얻는 물성	분리일(work of separation) $W_{sep} = (E_{슬래브} + E_{바인더} - E_{계면}) / A$ · 수분 조건 비교는 6주차 보충

계면 계산은 벌크와 달리 z 방향 대칭 없음 — 압력 제어도 x · y 에만 적용

4-5 · REAL CELL

플랫폼의 층상 셀 — 층마다 다른 힘장



평형화한 부모 셀 · SiO_2 건조 표면 · 293 K · 23,124 원자 (paper10 Fig. 2)

바인더 층

AAA1 216분자 · GAFF2 + AM1-BCC 전하

광물 슬래브

석영 · ClayFF 전하와 LJ (12-6)

두 층 사이

이종쌍은 Lorentz–Berthelot 조합 규칙(combining rule) · 장거리 쿨롱은 PPPM

경계 p p f · 인장은 z 방향 — 분리일 · 계면 인장강도 산출은 6주차 보충

다층 셀 조립 — 명령 순서

in.interface

```
units       real
atom_style  full
boundary    p p f

read_data   slab.data
change_box  all z final -10 140 units box

read_data   binder.data add append &
            offset 12 18 24 30 0 &
            shift 0.0 0.0 42.0

kspace_modify slab 3.0
fix   frz   bottom setforce 0.0 0.0 0.0
fix   nvt   mobile nvt temp 298 298 100
```

01 경계조건을 `p p f` 로 설정 — z 는 비주기

02 슬래브 읽기 후 상자를 z 로 확장 → 바인더 · 진공 공간 확보

03 두 번째 data 를 `add append` 로 병합 — 타입 번호 중복 방지는 `offset`, 위치는 `shift`

04 슬래브 보정 `kspace_modify slab` · 바닥층 고정 · 이동 가능한 원자만 NVT

05 접촉 후 NVT 로 밀착, 이후 x · y 만 압력 제어(`npt ... x 1.0 1.0 1000 y 1.0 1.0 1000`)

합치기 전 두 블록의 힘장 계수가 같은 규칙(mixing · special_bonds)을 쓰는지 반드시 확인

부착일 계산과 수분 조건 비교는 6주차 보충 — 여기서는 셀 생성까지

PRACTICE

실습 · 바인더 셀 평형화

PRACTICE

01 3주차 data 파일로 입력 스크립트를 작성

02 최소화 → 어닐링 → 298 K NPT 평형화를 실행

03 밀도 · 부피 시계열로 평형 구간을 판정

04 Packmol 로 만든 셀과 평형화 결과를 같은 폴더에 정리 (실험 ID · 입력 해시는 13주차 보충 A 13-S3 재현성 규약 적용)

DELIVERABLE

입력 스크립트 · log · thermo CSV · 최종 구조 · restart 파일

평형 판정 그래프 1장과 평형 밀도 (평균 ± 표준오차)

같은 폴더를 다른 사람이 받아 그대로 재실행할 수 있어야 합격

퀴즈 4주차

SUMMARY

4주차 정리

01 LAMMPS 는 패키지로 확장되는 고전 MD 엔진 — CPU · GPU · KOKKOS 빌드 간 물리 동일

02 입력 스크립트는 units · atom_style · pair_style · fix · run 의 다섯 블록으로 판독

03 data 파일 = 원자 좌표 · 토폴로지 · 타입 번호 · 의미는 힘장 계수가 결정

04 초기 구조는 RDKit · Packmol · Moltemplate 를 이어 붙여 목표 조성대로 생성 — 낮은 밀도로 패킹 · 검사 통과 셀만 사용

05 부착 · 계면 연구용 다층 셀은 경계조건 · 진공층 · 슬래브 고정이 핵심

퀴즈 4주차

다음 주차 — 자동화 워크플로우와 다중 힘장 스택

APPX

부록 · 참고 자료

본강의 흐름에서 빠진 슬라이드와 원문 보존

APPENDIX · ORIGINAL TEXT

이전 구성의 원문 문구

01 GPU 빌드 · 정밀도 · 검증

02 부록 LAMMPS · Packmol 설치를 요청하는 문장

03 설치를 Claude Code 에 맡길 때의 프롬프트